AI201 PA 4 Report
Comparison of Boosted Perceptrons and SVM

Deomary Angelo Franco (2023-22049)

Date of Submission: April 23, 2025

1. INTRODUCTION

In machine learning, there is the concept of weak and strong learners. Weak learners are models that are considered "only slightly better" than randomly guessing while strong learners are models that can generalize well when trained with enough data. Examples of weak learners are perceptron classifiers, decision stumps, and decision trees while strong learners are Support Vector Machines (SVM) and Artificial Neural Networks. While strong learners are the initial choice for modern problems, there is the technique of "boosting" the weak learners such that they can rival the learning capabilities of strong learners. An example of this is the Adaboost or Adaptive Boosting technique. Adaboost learns by developing multiple weak learners, with each learner focusing on the misclassifications of the prior learner, and then having a weighted sum of all the weak learners. This can be thought of as "voting" but with some votes being more impactful. Since weak learners are relatively simple, training multiple of these is not computationally expensive and is feasible and viable for most problems. With this in mind, could these boosted weak learners keep up with the popular robust strong learners?

2. OBJECTIVES

The purpose of the paper is to show and compare Adaboost (using perceptron classifier) and Support Vector Machines for binary classification problems. To compare them, here are the items that will be tackled:

- Training and Test Accuracies vs K (number of weak estimators),
- Determine the kernel and hyperparameters for the Support Vector Machine that gives the best accuracy on training,
- Comparison of Adaboost and SVM in terms of training accuracy, test accuracy, training speed, test speed, and hyperparameter optimization.

3. METHODOLOGY

3.1 Banana Dataset and Splice Dataset

The first dataset used for the study is the Banana Dataset which is seen in Figure 1. The classes (+1 and -1) are shown also in the figure. The data can easily be distinguished and can be separated by some nonlinear hyperplane. For this case, 400 randomly selected points are taken as the training data while the remaining 4900 points will serve as the test data. The Splice dataset is the second dataset with 1000 training data points and 2175 test data points. This dataset has 60 features and similar to the first dataset, also has classes of +1 and -1. For this case however, the visualization is hard to perform without using advanced techniques such as dimensionality-reduction techniques, hence will not be shown.

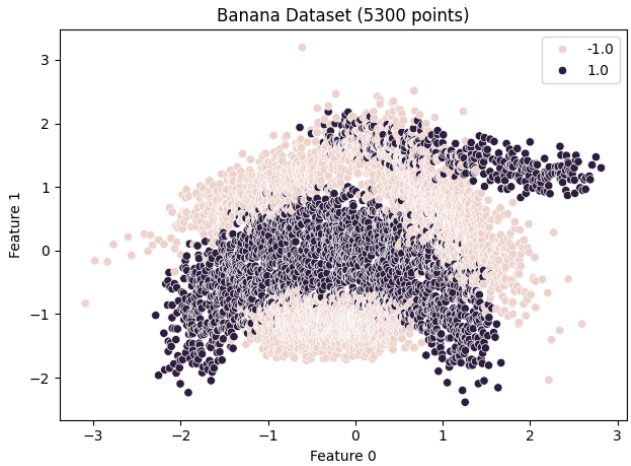

Figure 1: Shown is a scatterplot visualization of the banana dataset having 5,300 data points. It is easy to understand where the name of the dataset came from.

3.2 Adaboost with Perceptron Classifier

The Jupyter notebook [1] provides the implementation used in this study, while the document [2] outlines the algorithm specifications. The weak learner used is the Perceptron Classifier which uses the pocket algorithm. Here, the algorithm is trained in a way that maximizes the number of consecutive correct predictions. The weights that give the highest consecutive correct guess are the parameters learned by this type of weak learner.

Using the weak learner, Adaboost "boosts" this weak learner by generating K number of learners, with each successive learner being trained more on the incorrect predictions from the previous learner. The final prediction is then the weighted sum of all the learners used. The Adaboost model will be trained on $K = 1000$. The good thing here is that if we train the model using this hyperparameter, then the model is already trained for all $K < 1000$ since the learned parameters are also a subset of the $K = 1000$. For example, if it is of interest to have a model of having only $K = 200$, then only get the learned hyperparameters for the 200 weak learners (i.e. no need to retrain Adaboost using this hyperparameter) for inference.

3.3 Support Vector Machine

Implementing the SVM in the notebook is done by using [3] which has built-in codes for performing the classification problem in different languages. For this case, the Python version is used. SVM is performed by separating the data points by some hyperplane. However, a hyperplane may not be enough to separate nonlinear data, hence the usage of the kernel trick. Luckily, the libsvm package of [3] has this covered. Determining the hyperparameters for the optimized model can also be performed using the same package. For this method, ensure that the data files are transformed to .libsvm format to avoid any issues.

3.4 Testing and Evaluation

The two models, Adaboost and SVM, are tested on each dataset, Banana and Splice. While the default versions of the models may be mediocre, each model will be searched with the optimal hyperparameters that will give the best accuracy on the training set. Once the hyperparameters are determined, these will then be used on the training dataset. Once the model is trained, it is then evaluated on the test dataset. The accuracy will then be compared between the two models, both performed on the two datasets. Metrics that will be looked at are the training accuracy, test accuracy, training speed, and test speed. For the training speed, the time spent searching for the best hyperparameters will also be considered since it is still technically "training" or developing the models.

4. EXPERIMENTAL RESULTS

After training and testing the Adaboost implementations, the accuracy for training and testing for the two datasets are displayed in Figure 2 and Figure 3 concerning the number of estimators.

The SVM hyperparameters that were discovered from the search that provided the best accuracy during training are shown in Table 1.

In Table 2, listed here are the different metrics using each model on each dataset. Here, train speed refers to the time spent on training the model using the training dataset on the final hyperparameters. However, one should also consider the metric Hyperparameter Search Speed as this is the time spent on determining the hyperparameter used to determine the final model.

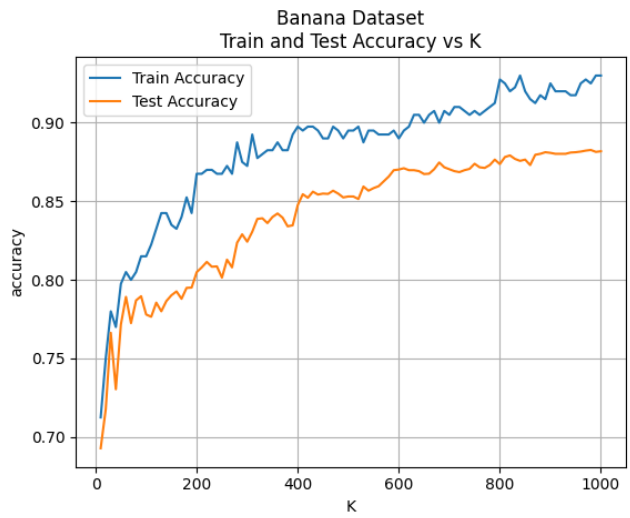

Figure 2: Shown is the training and test accuracy vs K for the Banana Dataset using Adaboost. As shown, the trend is that as you increase the number of estimators used, the accuracy also increases.

	Kernel	C	γ	CV	Acc. (%)
Banana	RBF	8	2	5	89.00
Splice	RBF	8	0.0078125	5	88.70

Table 1: Shown are the hyperparameters Kernel, C, and γ using cross-validation of 5 and the corresponding cross-validation accuracy for the two datasets used.

	Banana	Banana	Splice	Splice
	Adaboost	SVM	Adaboost	SVM
Train Acc. (%)	93.0000	91.0000	87.1000	100.0000
Test Acc. (%)	88.1837	88.9184	84.1379	89.7471
Train Speed (s)	39.6936	0.0031	40.4367	0.0934
Test Speed (s)	0.2074	0.0303	0.4358	0.1341
Search Speed (s)	39.6936	127.3918	40.4367	489.9821

Table 2: Shown here are the accuracies and speed in using Adabost and SVM for the different datasets. While SVM boasts a higher accuracy on test, the search speed is quite expensive.

5. ANALYSIS AND DISCUSSION OF RESULTS

In the figures, Figure 2 and Figure 3, both models have a trend that accuracy increases while the number of estimators increases. While oscillations are present, one can still generalize that with the higher number of weak learners (or estimators), the model does not yet overfit (at least for a model using perceptron as the base model), as presented by the accuracy plot. Hence, the selected hyperparameter for the models used in both datasets will be $K = 1000$. The convenient aspect here is that one can still utilize a model with a lower hyperparameter K since these are just stored in memory (unlike for the SVM models where a hyperparameter search meant retraining the data on different permutations).

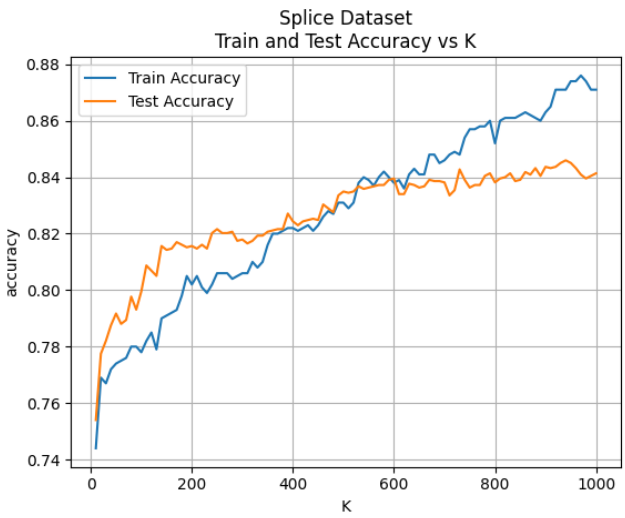

Figure 3: Shown is the training and test accuracy vs K for the Splice Dataset using Adaboost. Similar to the Banana dataset, the accuracy for the Splice Dataset also increases as the number of estimators increases. It can also be observed here that the test accuracy is also higher than the training accuracy for $K < 500$. This signifies that the model is underfitting for low number of estimators.

For the SVM model, it is shown that the best models use an RBF (Radial Basis Function). This can be expected since especially from the 2D plot in Figure 1, a linear hyperplane nor a polynomial hyperplane could not easily separate the data. When looking at Table 2, while SVM provides better generalization capabilities (especially when using the RBF Kernel), searching takes significant time, as observed from the Search Speed. Training and inference speed is still fast for small datasets, the main drawback is due to the larger space needed to be searched. Unlike Adaboost where only a certain K value is needed to be trained and the whole search result is already provided. It is still important to note training an instance of SVM is still faster than training an instance of Adaboost. Nonetheless, the test accuracy of using Adaboost and SVM for the Banana dataset shows comparable results, but SVM is the clear winner for the Splice Dataset when compared to Adaboost.

6. CONCLUSION

To summarize the study, one can see that SVM is clearly strong in multiple aspects. While this would be a sign of a better model, there is still the problem of how to reach the optimal state of the model–which can be achieved by hyperparameter optimization. In this aspect, due to a large search space, this is where Adaboost is the better choice.

For the Adaboost, it can be seen that if one increases the K, it can be thought that the model can still improve further, albeit at the brink of plateauing. If it is of interest to squeeze a higher accuracy, then a higher number of estimators would be great but at the cost of more computational resources and memory (i.e. diminishing returns).

Suggested improvements to this study would be to use this on another dataset that has significantly larger counts (i.e. stress test the models). It would also be worth checking other weak learners other than the perceptron classifier and seeing its impact since it is known that the RBF kernel for SVM is already robust.

7. REFERENCES

[1] D. A. B. Franco, "Jupyter notebook for programming assignment 4: Comparison of boosted perceptrons and svm," Course submission, AI201, University of the Philippines, 2025, attached with the report.

[2] P. C. Naval, "Programming Assignment 4: Comparison of Boosted Perceptrons and SVM," 2025, AI201 Course Material, University of the Philippines.

[3] C.-C. Chang and C.-J. Lin, "Libsvm: A library for support vector machines," *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, pp. 27:1–27:27, 2011, version 3.17 used in this study.